What is IAM?

PDF

RSS

Markdown

Focus mode

AWS Identity and Access Management (IAM) is a web service that helps you securely control access to AWS resources. With IAM, you can manage permissions that control which AWS resources users can access. You use IAM to control who is authenticated (signed in) and authorized (has permissions) to use resources. IAM provides the infrastructure necessary to control authentication and authorization for your AWS accounts.

Identities

When you create an AWS account, you begin with one sign-in identity called the AWS account *root user* that has complete access to all AWS services and resources. We strongly recommend that you don't use the root user for everyday tasks. For tasks that require root user credentials, see Tasks that require root user credentials in the *IAM User Guide*.

Use IAM to set up other identities in addition to your root user, such as administrators, analysts, and developers, and grant them access to the resources they need to succeed in their tasks.

Access management

After a user is set up in IAM, they use their sign-in credentials to authenticate with AWS. Authentication is provided by matching the sign-in credentials to a principal (an IAM user, AWS STS federated principal, IAM role, or application) trusted by the AWS account. Next, a request is made to grant the principal access to resources. Access is granted in response to an authorization request if the user has been given permission to the resource. For example, when you first sign in to the console and are on the console Home page, you aren't accessing a specific service. When you select a service, the request for authorization is sent to that service and it looks to see if your identity is on the list of authorized users, what policies are being enforced to control the level of access granted, and any other policies that might be in effect. Authorization requests can be made by principals within your AWS account or from another AWS account that you trust.

Once authorized, the principal can take action or perform operations on resources in your AWS account. For example, the principal could launch a new Amazon Elastic Compute Cloud instance, modify IAM group membership, or delete Amazon Simple Storage Service buckets.

Tip

AWS Training and Certification provides a 10-minute video introduction to IAM:

Introduction to AWS Identity and Access Management.

Service availability

IAM, like many other AWS services, is eventually consistent. IAM achieves high availability by replicating data across multiple servers within Amazon's data centers around the world. If a request to change some data is successful, the change is committed and safely stored. However, the change must be replicated across IAM, which can take some time. Such changes include creating or updating users, groups, roles, or policies. We recommend that you do not include such IAM changes in the critical, high-availability code paths of your application. Instead, make IAM changes in a separate initialization or setup routine that you run less frequently. Also, be sure to verify that the changes have been propagated before production workflows depend on them. For more information, see Changes that I make are not always immediately visible.

Service cost information

AWS Identity and Access Management (IAM), AWS IAM Identity Center and AWS Security Token Service (AWS STS) are features of your AWS account offered at no additional charge. You are charged only when you access other AWS services using your IAM users or AWS STS temporary security credentials.

IAM Access Analyzer external access analysis is offered at no additional charge. However, you will incur charges for unused access analysis and customer policy checks. For a complete list of charges and prices for IAM Access Analyzer, see IAM Access Analyzer pricing.

For information about the pricing of other AWS products, see the Amazon Web Services pricing page.

Integration with other AWS services

IAM is integrated with many AWS services. For a list of AWS services that work with IAM and the IAM features the services support, see AWS services that work with IAM.

Why should I use IAM?

PDF

RSS

Markdown

Focus mode

AWS Identity and Access Management is a powerful tool for securely managing access to your AWS resources. One of the primary benefits of using IAM is the ability to grant shared access to your AWS account. Additionally, IAM allows you to assign granular permissions, enabling you to control exactly what actions different users can perform on specific resources. This level of access control is crucial for maintaining the security of your AWS environment. IAM also provides several other security features. You can add multi-factor authentication (MFA) for an extra layer of protection, and leverage identity federation to seamlessly integrate users from your corporate network or other identity providers. IAM also integrates with AWS CloudTrail, providing detailed logging and identity information to support auditing and compliance requirements. By taking advantage of these capabilities, you can help ensure that access to your critical AWS resources is tightly controlled and secure.

Shared access to your AWS account

You can grant other people permission to administer and use resources in your AWS account without having to share your password or access key.

Granular permissions

You can grant different permissions to different people for different resources. For example, you might allow some users complete access to Amazon Elastic Compute Cloud (Amazon EC2), Amazon Simple Storage Service (Amazon S3), Amazon DynamoDB, Amazon Redshift, and other AWS services. For other users, you can allow read-only access to just some Amazon S3 buckets, or permission to administer just some Amazon EC2 instances, or to access your billing information but nothing else.

Secure access to AWS resources for applications that run on Amazon EC2

You can use IAM features to securely provide credentials for applications that run on EC2 instances. These credentials provide permissions for your application to access other AWS resources. Examples include S3 buckets and DynamoDB tables.

Multi-factor authentication (MFA)

You can add two-factor authentication to your account and to individual users for extra security. With MFA you or your users must provide not only a password or access key to work with your account, but also a code from a specially configured device. If you already use a FIDO security key with other services, and it has an AWS supported configuration, you can use WebAuthn for MFA security. For more information, see Supported configurations for using passkeys and security keys

Identity federation

You can allow users who already have passwords elsewhere—for example, in your corporate network or with an internet identity provider—to access your AWS account. These users are granted temporary credentials that comply with IAM best practice recommendations. Using identity federation enhances the security of your AWS account.

Identity information for assurance

If you use AWS CloudTrail, you receive log records that include information about those who made requests for resources in your account. That information is based on IAM identities.

PCI DSS Compliance

IAM supports the processing, storage, and transmission of credit card data by a merchant or service provider, and has been validated as being compliant with Payment Card Industry (PCI) Data Security Standard (DSS). For more information about PCI DSS, including how to request a copy of the AWS PCI Compliance Package, see PCI DSS Level 1.

When do I use IAM?

AWS Identity and Access Management is a core infrastructure service that provides the foundation for access control based on identities within AWS. You use IAM every time you access your AWS account. The way you use IAM will depend on the specific responsibilities and job functions within your organization. Users of AWS services use IAM to access the AWS resources required for their day-to-day work, with administrators granting the appropriate permissions. IAM administrators, on the other hand, are responsible for managing IAM identities and writing policies to control access to resources. Regardless of your role, you interact with IAM whenever you authenticate and authorize access to AWS resources. This could involve signing in as an IAM user, assuming an IAM role, or leveraging identity federation for seamless access. Understanding the various IAM capabilities and use cases is crucial for effectively managing secure access to your AWS environment. When it comes to creating policies and permissions, IAM provides a flexible and granular approach. You can define trust policies to control which principals can assume a role, in addition to identity-based policies that specify the actions and resources a user or role can access. By configuring these IAM policies, you can help ensure that users and applications have the appropriate level of permissions to perform their required tasks.

When you are performing different job functions

AWS Identity and Access Management is a core infrastructure service that provides the foundation for access control based on identities within AWS. You use IAM every time you access your AWS account.

How you use IAM differs, depending on the work that you do in AWS.

- Service user – If you use an AWS service to do your job, then your administrator provides you with the credentials and permissions that you need. As you use more advanced features to do your work, you might need additional permissions. Understanding how access is managed can help you request the right permissions from your administrator.
- Service administrator – If you're in charge of an AWS resource at your company, you probably have full access to IAM. It's your job to determine which IAM features and resources your service users should access. You must then submit requests to your IAM administrator to change the permissions of your service users. Review the information on this page to understand the basic concepts of IAM.
- IAM administrator – If you're an IAM administrator, you manage IAM identities and write policies to manage access to IAM.

When you are authorized to access AWS resources

Authentication is how you sign in to AWS using your identity credentials. You must be authenticated as the AWS account root user, an IAM user, or by assuming an IAM role.

You can sign in as a federated identity using credentials from an identity source like AWS IAM Identity Center (IAM Identity Center), single sign-on authentication, or Google/Facebook credentials. For more information about signing in, see How to sign in to your AWS account in the *AWS Sign-In User Guide*.

For programmatic access, AWS provides an SDK and CLI to cryptographically sign requests. For more information, see AWS Signature Version 4 for API requests in the *IAM User Guide*.

When you sign-in as an IAM user

An *IAM user* is an identity with specific permissions for a single person or application. We recommend using temporary credentials instead of IAM users with long-term credentials. For more information, see Require human users to use federation with an identity provider to access AWS using temporary credentials in the *IAM User Guide*.

An *IAM group* specifies a collection of IAM users and makes permissions easier to manage for large sets of users. For more information, see Use cases for IAM users in the *IAM User Guide*.

When you assume an IAM role

An *IAM role* is an identity with specific permissions that provides temporary credentials. You can assume a role by switching from a user to an IAM role (console) or by calling an AWS CLI or AWS API operation. For more information, see Methods to assume a role in the *IAM User Guide*.

IAM roles are useful for federated user access, temporary IAM user permissions, cross-account access, cross-service access, and applications running on Amazon EC2. For more information, see Cross account resource access in IAM in the *IAM User Guide*.

When you create policies and permissions

You grant permissions to a user by creating a policy, which is a document that lists the actions that a user can perform and the resources those actions can affect. Any actions

or resources that are not explicitly allowed are denied by default. Policies can be created and attached to principals (users, groups of users, roles assumed by users, and resources).

You can use these policies with an IAM role:

- **Trust policy** – Defines which principal can assume the role, and under which conditions. A trust policy is a specific type of resource-based policy for IAM roles. A role can have only one trust policy.
- **Identity-based policies (inline and managed)** – These policies define the permissions that the user of the role is able to perform (or is denied from performing), and on which resources.

Use the Example IAM identity-based policies to help you define permissions for your IAM identities. After you find the policy that you need, choose view the policy to view the JSON for the policy. You can use the JSON policy document as a template for your own policies.

Note

If you are using IAM Identity Center to manage your users, you assign permission sets in IAM Identity Center instead of attaching a permissions policy to a principal. When you assign a permission set to a group or user in AWS IAM Identity Center, IAM Identity Center creates corresponding IAM roles in each account, and attaches the policies specified in the permission set to those roles. IAM Identity Center manages the role, and allows the authorized users you've defined to assume the role. If you modify the permission set, IAM Identity Center ensures that the corresponding IAM policies and roles are updated accordingly.

For more information about IAM Identity Center, see What is IAM Identity Center? in the *AWS IAM Identity Center User Guide*.

How do I manage IAM?

PDF

RSS

Markdown

Focus mode

Managing AWS Identity and Access Management within an AWS environment involves leveraging a variety of tools and interfaces. The most common method is through the

AWS Management Console, a web-based interface that allows you to perform a wide range of IAM administrative tasks, from creating users and roles to configuring permissions.

For users more comfortable with command line interfaces, AWS provides two sets of command line tools - the AWS Command Line Interface and the AWS Tools for Windows PowerShell. These allow you to issue IAM-related commands directly from the terminal, often more efficiently than navigating the console. Additionally, AWS CloudShell enables you to run CLI or SDK commands directly from your web browser, using the permissions associated with your console sign-in.

Beyond the console and command line, AWS offers Software Development Kits (SDKs) for various programming languages, enabling you to integrate IAM management functionality directly into your applications. Alternatively, you can access IAM programmatically using the IAM Query API, which allows you to issue HTTPS requests directly to the service. Leveraging these different management approaches provides you with the flexibility to incorporate IAM into your existing workflows and processes.

Use the AWS Management Console

The AWS Management Console is a web application that comprises and refers to a broad collection of service consoles for managing AWS resources. When you first sign in, you see the console home page. The home page provides access to each service console and offers a single place to access the information for performing your AWS related tasks. Which services and applications are available to you after signing in to the console depend on which AWS resources you have permission to access. You can be granted permissions to resources either through assuming a role, being a member of a group that has been granted permissions, or being explicitly granted permission. For a stand-alone AWS account, the root user or IAM administrator configures access to resources. For AWS Organizations, the management account or delegated administrator configures access to resources.

If you plan to have people using the AWS Management Console to manage AWS resources, we recommend configuring users with temporary credentials as a security best practice. IAM users that have assumed a role, federated principals, and users in IAM Identity Center have temporary credentials, while the IAM user and root user have long-term credentials. Root user credentials provide full access to the AWS account, while other users have credentials that provide access to the resources granted them by IAM policies.

The sign-in experience is different for the different types of AWS Management Console users.

- IAM users and the root user sign-in from the main AWS sign-in URL (https://signin.aws.amazon.com). Once they sign in they have access to the resources in the account to which they have been granted permission.

To sign in as the root user you must have the root user email address and password.

To sign in as an IAM user you must have the AWS account number or alias, the IAM user name, and the IAM user password.

We recommend that you restrict IAM users in your account to specific situations that require long-term credentials, such as for emergency access, and that you use the root user only for tasks that require root user credentials.

For convenience, the AWS sign-in page uses a browser cookie to remember the IAM user name and account information. The next time the user goes to any page in the AWS Management Console, the console uses the cookie to redirect the user to the account sign-in page.

Sign out of the console when you finish your session to prevent reuse of your previous sign in.

- IAM Identity Center users sign in using a specific AWS access portal that's unique to their organization. Once they sign in they can choose which account or application to access. If they choose to access an account, they choose which permission set they want to use for the management session.
- OIDC and SAML federated principals managed in an external identity provider linked to an AWS account sign-in using a custom enterprise access portal. The AWS resources available to users are dependent upon the policies selected by their organization.

Note

To provide an additional level of security, root user, IAM users, and users in IAM Identity Center can have multi-factor authentication (MFA) verified by AWS before granting access to AWS resources. When MFA is enabled, you must also have access to the MFA device to sign in.

To learn more about how different users sign-in to the management console, see Sign in to the AWS Management Console in the *AWS Sign-In User Guide*.

AWS Command Line Tools

You can use the AWS command line tools to issue commands at your system's command line to perform IAM and AWS tasks. Using the command line can be faster

and more convenient than the console. The command line tools are also useful if you want to build scripts that perform AWS tasks.

AWS provides two sets of command line tools: the AWS Command Line Interface (AWS CLI) and the AWS Tools for Windows PowerShell. For information about installing and using the AWS CLI, see the AWS Command Line Interface User Guide. For information about installing and using the Tools for Windows PowerShell, see the AWS Tools for PowerShell User Guide.

After signing in to the console, you can use AWS CloudShell from your browser to run CLI or SDK commands. The permissions for accessing AWS resources are based on the credentials you used to sign-in to the console. Depending on your experience, you may find the CLI to be a more efficient method of managing your AWS account. For more information, see Use AWS CloudShell to work with AWS Identity and Access Management

AWS Command Line Interface (CLI) and Software Development Kits (SDKs)

IAM Identity Center and IAM users use different methods to authenticate their credentials when they authenticate through the CLI or the application interfaces (APIs) in the associated SDKs.

Credentials and configuration settings are located in multiple places, such as the system or user environment variables, local AWS configuration files, or explicitly declared on the command line as a parameter. Certain locations take precedence over others.

Both IAM Identity Center and IAM provide access keys that can be used with the CLI or SDK. IAM Identity Center access keys are temporary credentials that can be automatically refreshed and are recommended over the long-term access keys associated with IAM users.

To manage your AWS account using the CLI or SDK you can use AWS CloudShell from your browser. If you use CloudShell to run CLI or SDK commands you must first sign-in to the console. The permissions for accessing AWS resources are based on the credentials you used to sign-in to the console. Depending on your experience, you may find the CLI to be a more efficient method of managing your AWS account.

For application development, you can download the CLI or SDK to your computer and sign-in from the command prompt or a Docker window. In this scenario, you configure authentication and access credentials as part of the CLI script or SDK application. You

can configure programmatic access to resources in different ways, depending on the environment and the access available to you.

- Recommended options for authenticating local code with AWS service are IAM Identity Center and IAM Roles Anywhere
- Recommended options for authenticating code running within an AWS environment are to use IAM roles or use IAM Identity Center credentials.

When signing in using the AWS access portal, you can get short-term credentials from the start page where you choose your permission set. These credentials have a defined duration and don't automatically refresh. If you want to use these credentials, after signing in to the AWS portal, choose the AWS account and then choose the permissions set. Select **Command line or programmatic access** to view the options you can use to access AWS resources programmatically or from the CLI. For more information about these methods, see Getting and refreshing temporary credentials in the *IAM Identity Center User Guide*. These credentials are often used during application development to quickly test code.

We recommend using IAM Identity Center credentials that automatically refresh when automating access to your AWS resources. If you have configured users and permission sets in IAM Identity Center you use the `aws configure sso` command to use a command-line wizard that will help you identify the credentials available to you and store them in a profile. For more information about configuring your profile, see Configure your profile with the aws configure sso wizard in the *AWS Command Line Interface User Guide for Version 2*.

Note

Many sample applications use long-term access keys associated with IAM users or root user. You should only use long-term credentials within a sandbox environment as part of a learning exercise. Review the alternatives to long-term access keys and plan to transition your code to use alternative credentials, such as IAM Identity Center credentials or IAM roles, as soon as possible. After transitioning your code, delete the access keys.

To learn more about configuring the CLI, see Install or update the latest version of the AWS CLI in the *AWS Command Line Interface User Guide for Version 2* and Authentication and access credentials in the *AWS Command Line Interface User Guide*

To learn more about configuring the SDK, see IAM Identity Center authentication in the *AWS SDKs and Tools Reference Guide* and IAM Roles Anywhere in the *AWS SDKs and Tools Reference Guide*.

Use the AWS SDKs

AWS provides SDKs (software development kits) that consist of libraries and sample code for various programming languages and platforms (Java, Python, Ruby, .NET, iOS, Android, etc.). The SDKs provide a convenient way to create programmatic access to IAM and AWS. For example, the SDKs take care of tasks such as cryptographically signing requests, managing errors, and retrying requests automatically. For information about the AWS SDKs, including how to download and install them, see the Tools for Amazon Web Services page.

Use the IAM Query API

You can access IAM and AWS programmatically by using the IAM Query API, which lets you issue HTTPS requests directly to the service. When you use the Query API, you must include code to digitally sign requests using your credentials. For more information, see Calling the IAM API using HTTP query requests and the IAM API Reference.

How IAM works

PDF

RSS

Markdown

Focus mode

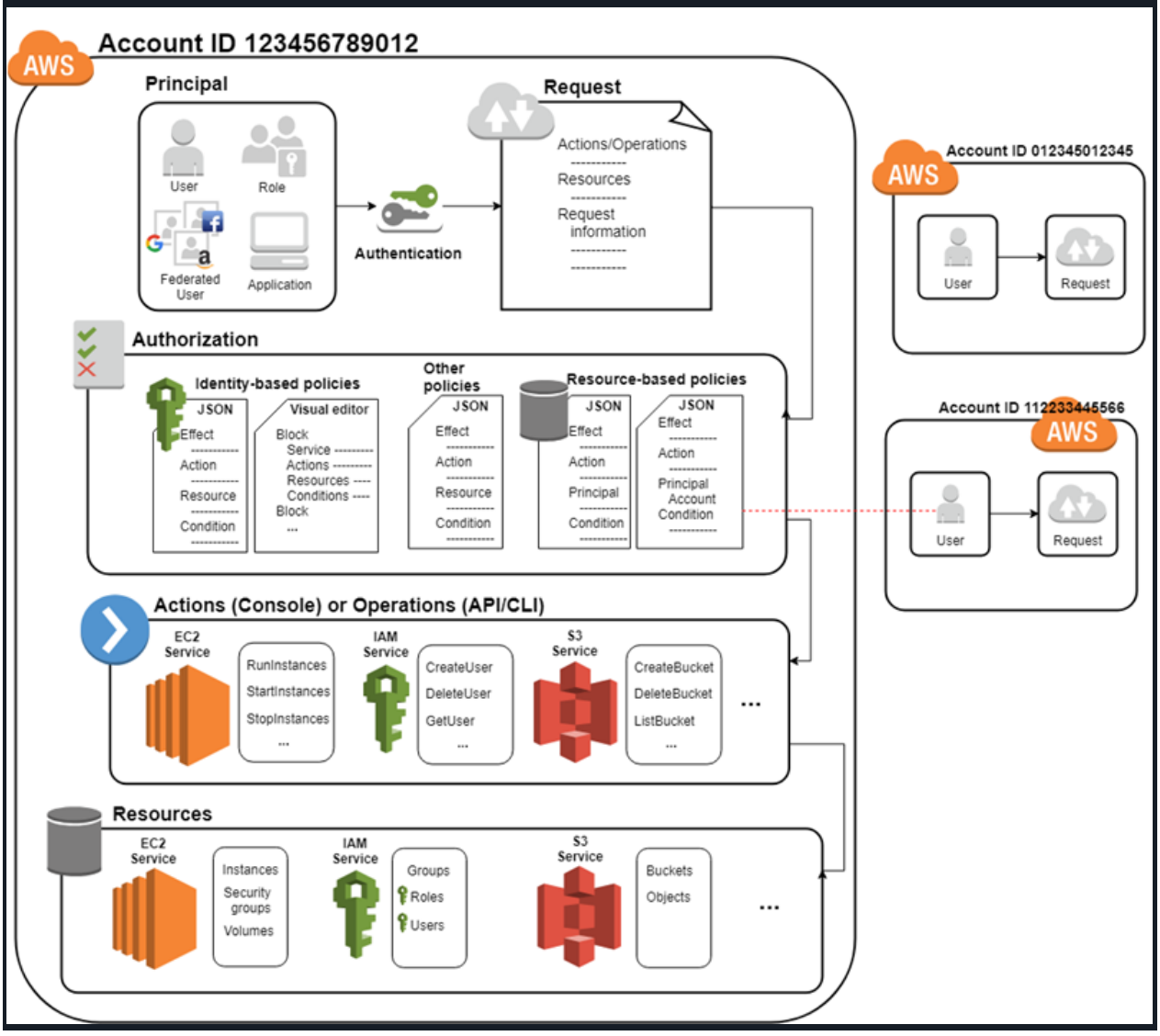
AWS Identity and Access Management provides the infrastructure necessary to control authentication and authorization for your AWS account.

First, a human user or an application uses their sign-in credentials to authenticate with AWS. IAM matches the sign-in credentials to a principal (an IAM user, AWS STS federated user principal, IAM role, or application) trusted by the AWS account and authenticates the principal to access AWS.

Next, IAM makes a request to grant the principal access to resources. IAM grants or denies access in response to an authorization request. For example, when you first sign in to the console and are on the console Home page, you aren't accessing a specific service. When you select a service, you send an authorization request to IAM for that service. IAM verifies that your identity is on the list of authorized users, determines what policies control the level of access granted, and evaluates any other policies that might

be in effect. Principals within your AWS account or from another AWS account that you trust can make authorization requests.

Once authorized, the principal can perform actions or operations on resources in your AWS account. For example, the principal could launch a new Amazon Elastic Compute Cloud instance, modify IAM group membership, or delete Amazon Simple Storage Service buckets. The following diagram illustrates this process through the IAM infrastructure:



Components of a request

When a principal tries to use the AWS Management Console, the AWS API, or the AWS CLI, that principal sends a *request* to AWS. The request includes the following information:

- **Actions or operations** – The actions or operations that the principal wants to perform, such as an action in the AWS Management Console, or an operation in the AWS CLI or AWS API.
- **Resources** – The AWS resource object upon which the principal requests to perform an action or operation.
- **Principal** – The person or application that used an entity (user or role) to send the request. Information about the principal includes the permission policies.
- **Environment data** – Information about the IP address, user agent, SSL enabled status, and the timestamp.
- **Resource data** – Data related to the resource requested, such as a DynamoDB table name or a tag on an Amazon EC2 instance.

AWS gathers the request information into a *request context*, which IAM evaluates to authorize the request.

How principals are authenticated

A principal signs in to AWS using their credentials which IAM authenticates to permit the principal to send a request to AWS. Some services, such as Amazon S3 and AWS STS, allow specific requests from anonymous users. However, they're the exception to the rule. Each type of user goes through authentication.

- **Root user** – Your sign in credentials used for authentication are the email address you used to create the AWS account and the password you specified at that time.
- **Federated principal** – Your identity provider authenticates you and passes your credentials to AWS, you don't have to sign-in directly to AWS. Both IAM Identity Center and IAM support identity federation.
- **Users in AWS IAM Identity Center directory***(not federated)*– Users created directly in the IAM Identity Center default directory sign in using the AWS access portal and provide your username and password.
- **IAM user** – You sign-in by providing your account ID or alias, your username, and password. To authenticate workloads from the API or AWS CLI, you might

use temporary credentials through assuming a role or you might use long-term credentials by providing your access key and secret key.

To learn more about the IAM entities, see IAM users and IAM roles.

AWS recommends that you use multi-factor authentication (MFA) with all users to increase the security of your account. To learn more about MFA, see AWS Multi-factor authentication in IAM.

Authorization and permission policy basics

Authorization refers to the principal having the required permissions to complete their request. During authorization, IAM identifies the policies that apply to the request using values from the request context. It then uses the policies to determine whether to allow or deny the request. IAM stores most permission policies as JSON documents that specify the permissions for principal entities.

There are several types of policies that can affect an authorization request. To provide your users with permissions to access the AWS resources in your account, you can use identity-based policies. Resource-based policies can grant cross-account access. To make a request in a different account, a policy in the other account must allow you to access the resource *and* the IAM entity that you use to make the request must have an identity-based policy that allows the request.

IAM checks each policy that applies to the context of your request. IAM policy evaluation uses an *explicit deny*, which means that if a single permissions policy includes a denied action, IAM denies the entire request and stops evaluating. Because requests are *denied by default*, the applicable permissions policies must allow every part of your request for IAM to authorize your request. The evaluation logic for a request within a single account follows these basic rules:

- By default, all requests are denied. (In general, requests made using the AWS account root user credentials for resources in the account are always allowed.)
- An explicit allow in any permissions policy (identity-based or resource-based) overrides this default.
- The existence of an AWS Organizations service control policy (SCP) or resource control policy (RCP), IAM permissions boundary, or a session policy overrides the allow. If one or more of these policy types exists, they must all allow the request. Otherwise, it's implicitly denied. For more information on SCPs and RCPs, see Authorization policies in AWS Organizations in the *AWS Organizations User Guide*.

- An explicit deny in any policy overrides any allows in any policy.

To learn more, see Policy evaluation logic.

After IAM authenticates and authorizes the principal, IAM approves the actions or operations in their request by evaluating the permission policy that applies to the principal. Each AWS service defines the actions (operations) they support, and include things that you can do to a resource, such as viewing, creating, editing, and deleting that resource. The permission policy that applies to the principal must include the necessary actions to perform an operation. To learn more about how IAM evaluates permission policies, see Policy evaluation logic.

The service defines a set of actions that a principal can perform on each resource. When creating permission policies, make sure to include the actions that you want the user to be able to perform. For example, IAM supports over 40 actions for a user resource, including the following basic actions:

- `CreateUser`
- `DeleteUser`
- `GetUser`
- `UpdateUser`

In addition, you can specify conditions in your permission policy that provide access to resources when the request meets the specified conditions. For example, you might want a policy statement to take effect after a specific date or to control access when a specific value appears in an API. To specify conditions, you use the Condition element of a policy statement.

After IAM approves the operations in a request, the principal can work with the related resources in your account. A resource is an object that exists within a service. Examples include an Amazon EC2 instance, an IAM user, and an Amazon S3 bucket. If the principal creates a request to perform an action on a resource that isn't included in the permission policy, the service denies the request. For example, if you have permission to delete an IAM role but request to delete an IAM group, the request fails if you don't have permission to delete IAM groups. To learn more about which actions, resources, and condition keys the different AWS services support, see Actions, Resources, and Condition Keys for AWS Services.